Multi-Agent Reinforcement Learning with PPO and LLM-Assisted Guidance on PettingZoo Simple Spread

Abstract

This project studies cooperative multi-agent reinforcement learning in PettingZoo `simple_spread_v3`. Three homogeneous agents must spread over three landmarks while avoiding collisions. We compare three methods under the same lightweight experimental budget: Independent PPO (IPPO), a MAPPO-style variant with decentralized actors and a centralized critic, and LLM-guided IPPO. The LLM-guided variant uses sparse high-level agent-to-landmark assignments and converts them into distance-based reward shaping, so the language model acts as a strategic coordinator rather than a low-level controller. Each method is trained for 500 episodes under seeds 0, 1, and 2. Under this budget, all methods learn comparable behavior. MAPPO has the best independent evaluation return and coverage distance by a small margin, while IPPO and LLM-guided IPPO are very close. The main finding is that the implemented pipeline is reproducible and that sparse high-level guidance is feasible, but the lightweight training budget is not enough to establish a statistically strong winner.

1. Introduction and Problem Setting

Cooperative multi-agent reinforcement learning (MARL) studies how multiple decision makers can learn coordinated behavior from shared experience. The problem is harder than single-agent RL because each agent's learning changes the data distribution seen by the others, and because reward credit must be assigned across agents that may only observe part of the global state. This project uses the PettingZoo Multi-Particle Environment (MPE) task `simple_spread_v3`, a standard benchmark for testing decentralized cooperation.

In Simple Spread, three agents move in a continuous 2D world containing three landmarks. The team objective is to cover all landmarks while minimizing collisions. Good behavior requires both coverage and coordination: if all agents chase the same landmark, the team receives poor coverage; if agents cross paths or crowd together, collision penalties reduce performance.

The task can be formulated as a decentralized partially observable Markov decision process (Dec-POMDP). The global state consists of the positions and velocities of all agents and landmarks in the MPE simulator. Each agent receives an 18-dimensional local observation including its own velocity, its position, relative landmark positions, and relative positions of other agents. The continuous action space is a 5-dimensional Box action used by MPE to represent movement/no-op components that control 2D motion. The transition dynamics are given by the MPE physics simulator. The reward is cooperative: it penalizes landmark coverage distance and agent-agent collisions. The optimization objective is to maximize expected team return over an episode.

This setting is suitable for the course project because it tests core RL ideas, including policy-gradient optimization, baselines, multi-seed evaluation, and result analysis. It also leaves room for a lightweight extension: using a large language model or a structured fallback heuristic to provide interpretable high-level coordination guidance.

2. Approach and Methodology

All three methods share the same environment wrapper, logging pipeline, and neural network size. Policies use a Gaussian actor with two hidden layers of size 64 and Tanh activations. The actor outputs a distribution over the continuous action vector, and sampled actions are clipped to the environment bounds. Value functions are implemented with matching multilayer perceptrons. Advantages are computed with generalized advantage estimation (GAE), and training uses the PPO clipped surrogate objective.

Independent PPO

IPPO treats the agents as independent learners during credit assignment while sharing parameters across homogeneous agents. Each agent uses only its own local observation as actor input. The critic also receives the local observation and estimates that agent's value. Rollout data from all agents are flattened into one batch, which gives the shared actor and critic more samples per update while preserving decentralized execution. IPPO is the primary baseline because it is simple, reproducible, and directly tests whether local observations are sufficient for cooperative coverage.

MAPPO-Style PPO

The MAPPO-style method keeps the same decentralized actor as IPPO, so execution still requires only each agent's local observation. The difference is in the critic: instead of a local observation, the critic receives a centralized input built by concatenating all agents' observations. This gives the value function information about the team configuration during training, which can reduce credit-assignment difficulty. The method is "MAPPO-style" rather than a full reproduction of every MAPPO engineering detail; its purpose is to test whether centralized value estimation helps in this project pipeline.

LLM-Guided IPPO

The LLM-guided variant extends IPPO with sparse high-level guidance. The guidance module is configured for an OpenAI-compatible Qwen endpoint, but it is safe to run without an API key. If no key is found or a provider call fails, the module uses a deterministic nearest-unique-landmark heuristic. In the current experiment logs, guidance source is `heuristic`, with five guidance refreshes per seed. This matches the configured frequency of once every 100 episodes over 500 episodes.

The guidance signal is not used as a direct action. Instead, at selected episode boundaries, the coordinator assigns each agent to a distinct landmark based on the current state snapshot. The assignment is then converted into a small shaping reward:

```
r_i^guide = -alpha * distance(agent_i, assigned_landmark_i)
```

where `alpha = 0.05` in the configuration. The final reward used by the IPPO update is the environment reward plus this shaping term. This design keeps the RL agent responsible for low-level control while allowing high-level guidance to bias exploration toward better landmark coverage. The design also keeps API cost low because guidance is refreshed only sparsely rather than at every step.

3. Experiment Settings

The environment is PettingZoo `simple_spread_v3` with 3 agents, 3 landmarks, continuous actions, `max_cycles = 25`, and `local_ratio = 0.5`. All methods use the same actor and critic hidden sizes `[64, 64]`, learning rate `3e-4`, discount factor `gamma = 0.99`, GAE parameter `lambda = 0.95`, PPO clip coefficient `0.2`, value loss coefficient `0.5`, entropy coefficient `0.01`, max gradient norm `0.5`, 4 PPO update epochs, minibatch size 256, and initial log standard deviation `-0.5`.

The official lightweight experiment suite contains 3 methods x 3 seeds x 500 episodes:

- IPPO with seeds 0, 1, and 2.
- MAPPO-style PPO with seeds 0, 1, and 2.
- LLM-guided IPPO with seeds 0, 1, and 2.

Each training run saves a CSV log and the best checkpoint according to recent training return. After training, each checkpoint is evaluated deterministically for 10 episodes using separate evaluation seeds. Plots and summary tables are generated from the saved CSV files under `results/`.

The run was executed on macOS with Python 3.11.15 and PyTorch 2.11.0. CUDA was unavailable on this machine, and PyTorch selected Apple Metal (`mps`) for the configured training runs. The runtime summary is saved in `results/runtime_check.json`.

The evaluation metrics are:

- Episode return: total cooperative reward over an episode. Higher is better, and values are negative because the task is distance-penalty based.
- Coverage distance: mean distance from each landmark to its nearest agent. Lower is better.
- Collision rate: average number of collisions per step. Lower is better.

4. Results

Figure 1 shows training episode return over 500 episodes. Figures 2 and 3 show coverage distance and collision rate. The curves indicate that all three methods remain close under the lightweight training budget. The variance across seeds is larger than the small average differences among algorithms.

The final-window training summary, computed over the last 50 episodes of each method/seed run, is:

Method	Runs	Final Return	Coverage Distance	Collision Rate
IPPO	3	-81.777 +/- 5.078	0.826 +/- 0.118	0.027 +/- 0.010
LLM-guided IPPO	3	-81.889 +/- 5.276	0.830 +/- 0.122	0.026 +/- 0.010
MAPPO-style PPO	3	-82.190 +/- 6.021	0.841 +/- 0.149	0.026 +/- 0.012

On this training summary, IPPO has the best final return and coverage by a small amount, while LLM-guided IPPO and MAPPO have slightly lower collision rates. The differences are small relative to the standard deviations.

The deterministic checkpoint evaluation summary, averaged across the three saved checkpoints per method, is:

Method	Runs	Evaluation Return	Evaluation Coverage	Evaluation Collision
IPPO	3	-72.079 +/- 10.699	0.729 +/- 0.203	0.015 +/- 0.012
LLM-guided IPPO	3	-72.146 +/- 10.852	0.730 +/- 0.207	0.015 +/- 0.012
MAPPO-style PPO	3	-71.708 +/- 10.972	0.716 +/- 0.193	0.015 +/- 0.012

In deterministic evaluation, MAPPO-style PPO has the best mean return and coverage distance, but the margin is again small. IPPO and LLM-guided IPPO are almost tied. Collision rates are equal at the reported precision. These results support a conservative conclusion: all three methods are functional and comparable, but this lightweight run does not provide strong evidence that one method dominates the others.

Representative trajectory plots were generated for seed 0. They show agents moving in the 2D plane toward landmark regions, but the trajectories also reflect the limited training budget: behavior is not perfectly coordinated in every episode. This matches the quantitative results, where coverage distance improves but remains far from zero.

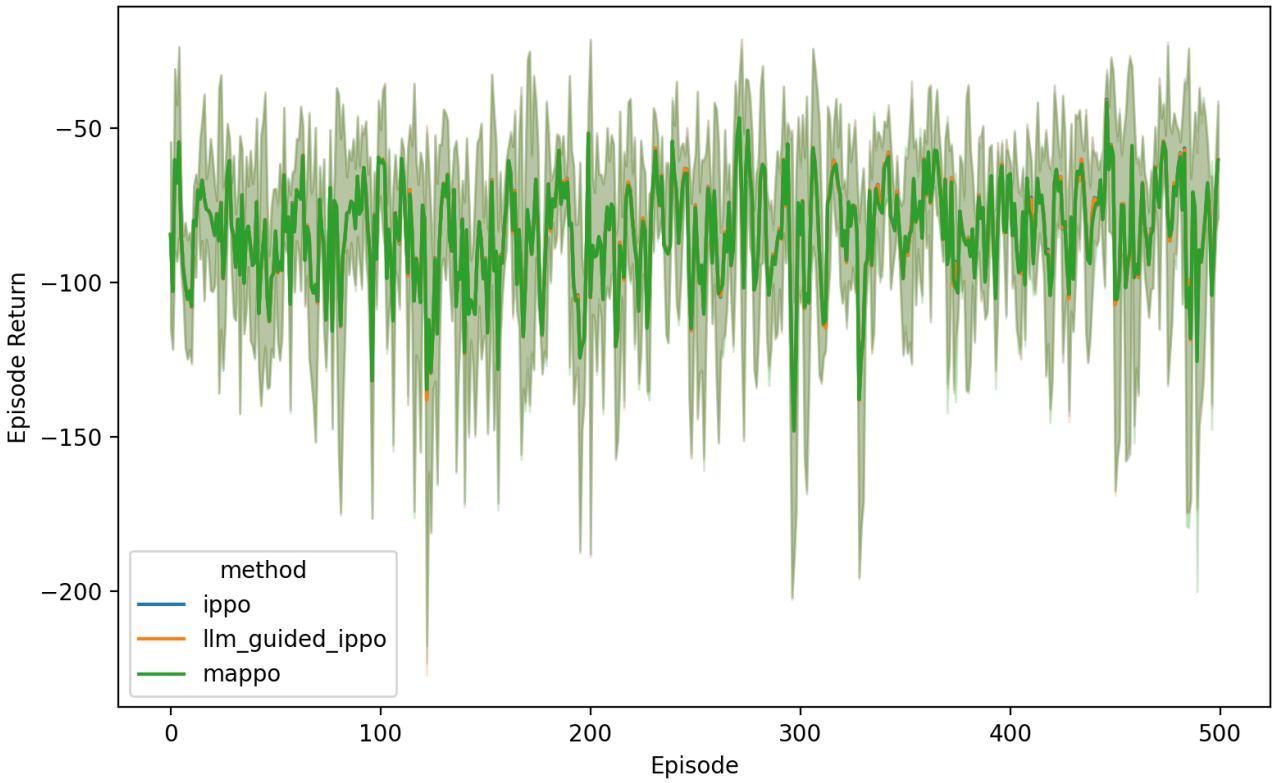

Figure 1. Training episode return over 500 episodes.

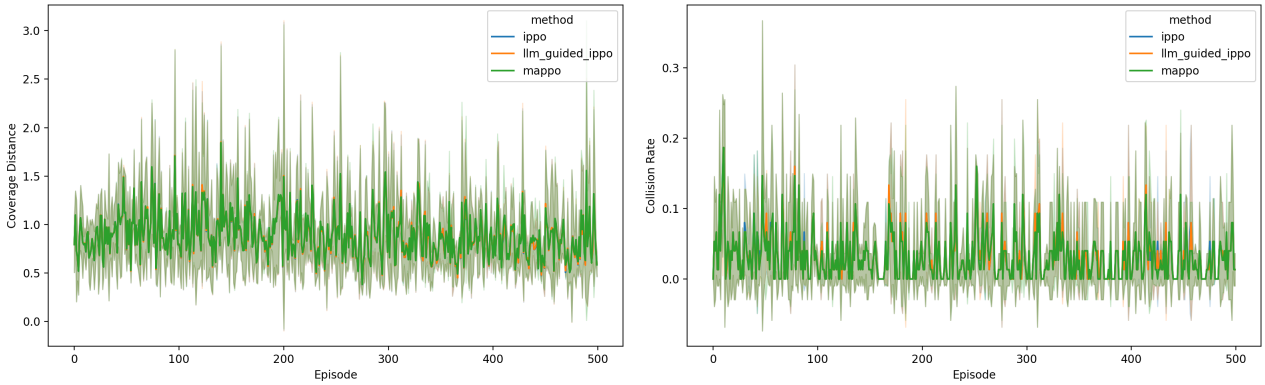

Figure 2. Coverage distance and collision rate during training.

method	runs	final_return	final_return_std	coverage_distance	coverage_distance_std	collision_rate	collision_rate_std
ippo	3	-81.777	5.078	0.826	0.118	0.027	0.010
llm_guided_ippo	3	-81.889	5.276	0.830	0.122	0.026	0.010
mappo	3	-82.190	6.021	0.841	0.149	0.026	0.012

method	runs	return mean+-std	coverage mean+-std	collision mean+-std
ippo	3	-72.079 +- 10.699	0.729 +- 0.203	0.015 +- 0.012
llm_guided_ippo	3	-72.146 +- 10.852	0.730 +- 0.207	0.015 +- 0.012
mappo	3	-71.708 +- 10.972	0.716 +- 0.193	0.015 +- 0.012

Figure 3. Final-window training summary and deterministic checkpoint evaluation summary.

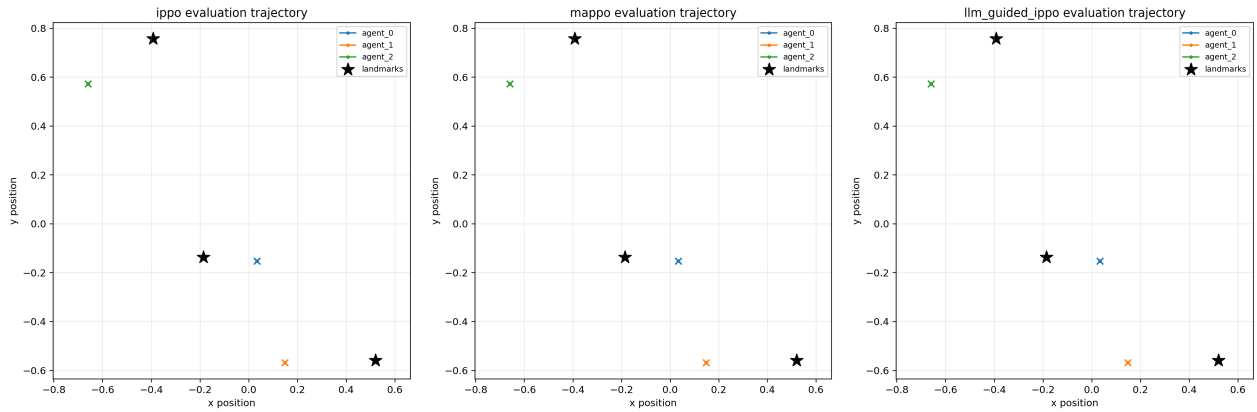

Figure 4. Representative deterministic evaluation trajectories for seed 0. Stars are landmarks.

5. Findings and Discussion

The first finding is that a compact PPO pipeline is sufficient to train non-trivial cooperative policies in Simple Spread. All three methods produce checkpoints that can be evaluated deterministically, and the learned policies achieve broadly similar returns.

The second finding is that the centralized critic in the MAPPO-style method does not dramatically change the outcome under 500 episodes, although it slightly improves deterministic evaluation return and coverage. A likely explanation is that the training budget is short and the task is small. With only three agents and three landmarks, the decentralized IPPO baseline already receives enough local information to learn partial coverage behavior. A longer run, stronger hyperparameter tuning, or a full MAPPO implementation with additional stabilization could make the centralized critic more valuable.

The third finding is that the LLM-guidance mechanism is operational but modest in effect. In this run, no external API key was used, so the Qwen-compatible provider fell back to the local heuristic. The mechanism still demonstrates how language-style high-level guidance can be translated into RL-compatible signals: the coordinator produces agent-landmark assignments, and the RL update receives distance-based shaping rewards. Because guidance is refreshed only five times per seed, its influence is sparse and intentionally gentle. This is safer than replacing the policy with LLM actions, but it also means the effect may be small under a short training horizon.

There are several limitations. First, 500 episodes is a lightweight budget, not a full convergence study. Second, the reported standard deviations show substantial seed sensitivity. Third, the LLM-assisted method was not evaluated with a live Qwen API call in this run; the current result should therefore be interpreted as a test of the guidance interface and heuristic fallback rather than evidence about language-model reasoning quality. Fourth, the

MAPPO-style baseline uses a centralized critic but does not include every engineering detail from full-scale MAPPO implementations.

Future work should extend training to 3000-5000 episodes, run more seeds, add ablations for reward-shaping coefficient and guidance frequency, and compare heuristic guidance with live Qwen guidance using the same seeds. It would also be useful to evaluate success rate or final landmark assignment quality in addition to return, coverage, and collisions.

6. Conclusion

This project implements and evaluates three PPO-based cooperative MARL methods on PettingZoo Simple Spread. The codebase supports config-driven training, checkpointing, deterministic evaluation, plotting, and artifact summarization. Under the 3 methods x 3 seeds x 500 episodes setting, IPPO, MAPPO-style PPO, and LLM-guided IPPO all run successfully and obtain comparable performance. MAPPO is slightly better in deterministic evaluation, while IPPO and LLM-guided IPPO are slightly better in the final training window. Because the differences are small relative to seed variance, the strongest conclusion is not that one method wins, but that the implemented experimental pipeline is reproducible and that sparse high-level guidance can be integrated into PPO without taking over low-level control.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. 2017.

[2] J. K. Terry et al. PettingZoo: Gym for Multi-Agent Reinforcement Learning. 2021.

[3] R. Lowe et al. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. 2017.

[4] C. Yu et al. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. 2022.